

Module 15 - chapter 6

Managing Host Key Checking and SSH keys

When Nana executes

-> `ansible all -i hosts -m ping`

She gets a prompt question regarding the Host Key checking (I don't and that is why my file `~/.ssh/known_hosts` is not getting the fingerprint stored) because it is in interactive mode.

```
nana@macbook /Users/nana
% ansible all -i hosts -m ping
The authenticity of host '157.230.29.113 (157.230.29.113)' can't be established.
ED25519 key fingerprint is SHA256:uIvpOCdDwx1a7D020ICHBgt/whUr2aHI/nEnU7Eazs.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
157.230.29.95 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
157.230.29.113 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
nana@macbook /Users/nana
% vim hosts
nana@macbook /Users/nana
% vim hosts
nana@macbook /Users/nana
% ansible droplet -i hosts -m ping
157.230.29.95 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

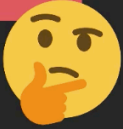
Host Key Checking

- ▶ Is enabled by default in Ansible
- ▶ It guards against server spoofing and man-in-the-middle attacks

If we want to automate configuration on the servers using ansible we want the Host Key check to be able to run automatically.

```
nana@macbook /Users/nana
% ansible all -i hosts -m ping
The authenticity of host '157.230.29.113 (157.230.29.113)' can't be established.
ED25519 key fingerprint is SHA256:uIvpOCdDwx1a7D020ICHBgt/whUr2aHI/nEnU7Eazs.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
157.230.29.95 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
157.230.29.113 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
nana@macbook /Users/nana
% vim hosts
nana@macbook /Users/nana
% vim hosts
nana@macbook /Users/nana
```

But we don't want to have this interactive mode!



So let's configure that

Authorized Keys & Known Hosts

There are two ways to handle ssh key check in general:

1. We create long running servers, meaning we create the servers and we will run them for a long time. For example the jenkins droplet
2. They re not ephemeral servers that get destroy all the time.

2 options based on:

- ▶ "long-lived" servers
- ▶ "ephemeral" or "temporary" servers



So for the long-lived servers we can do the ssh key check ONCE for each server from the control machine where Ansible is running and it is sorted, like I did with:

-> `ssh root@46.101.88.242`

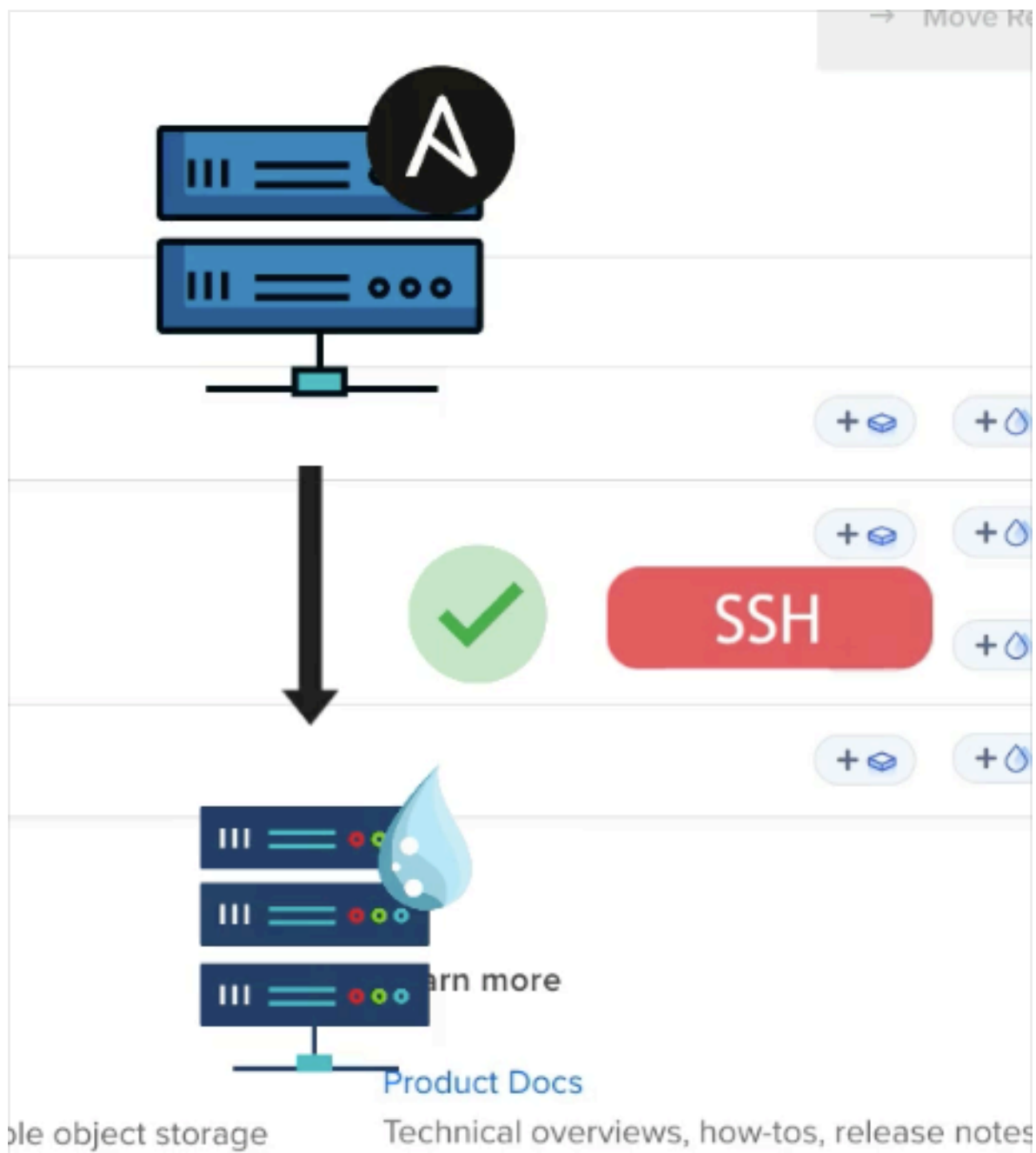
```
> ssh root@46.101.88.242
The authenticity of host '46.101.88.242 (46.101.88.242)' can't be established.
ED25519 key fingerprint is SHA256:KJN/tE0ekRlT7d0ARMBYIocZVPZdjwcW26hldGH6G0A.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '46.101.88.242' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

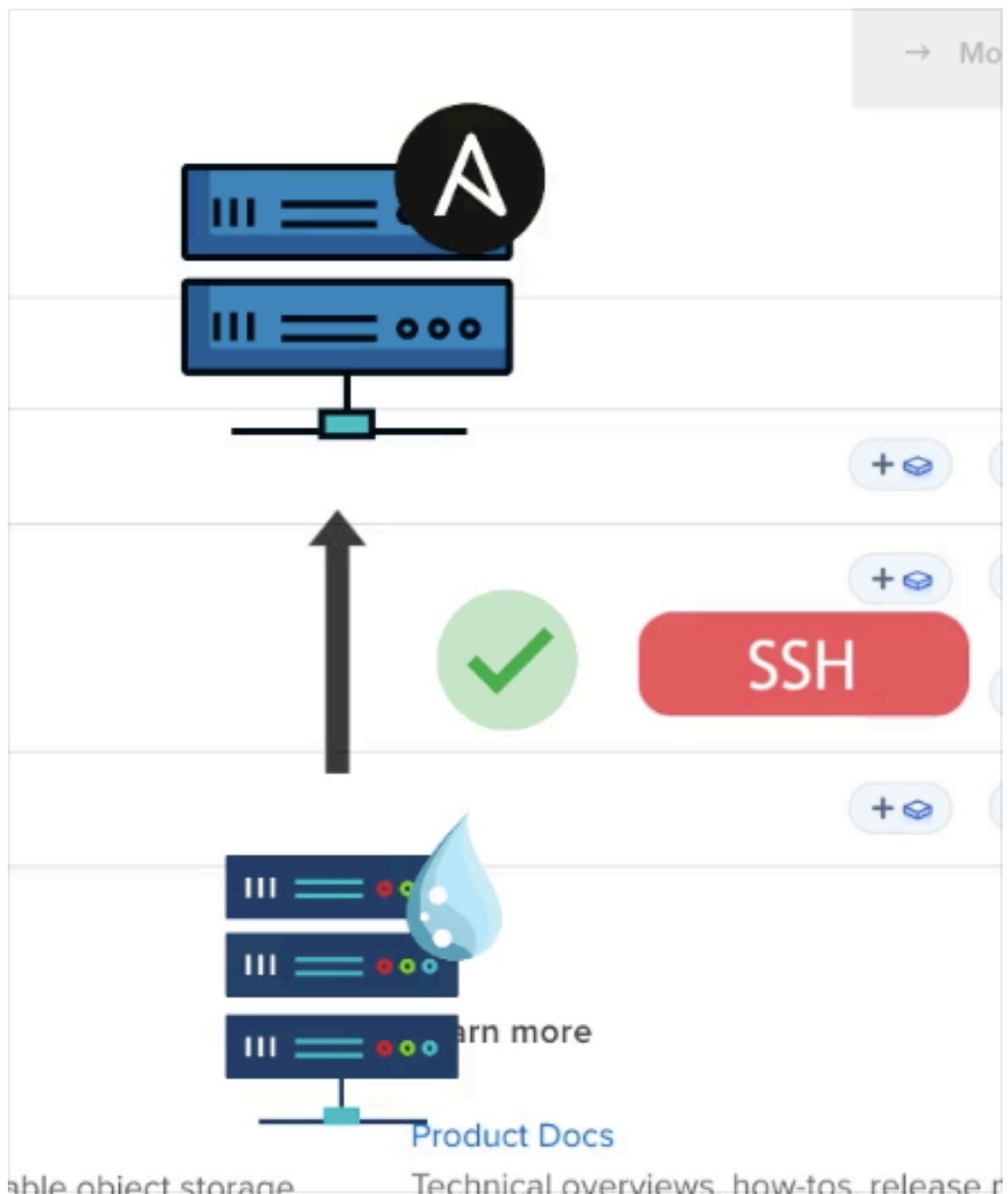
System information as of Tue Jul 14 10:38:12 UTC 2026
```

Or updating the `~/.ssh/known_hosts` file directly without the interactive prompt
-> `ssh-keyscan 46.101.80.55 >> ~/.ssh/known_hosts`

What we want is that both servers recognise (identify and authenticate) each other, so the droplet to recognise the control machine



and the control machine to recognise the droplet as a valid host.



A> So the `known_hosts` file must get the droplet information stored for my machine to recognise the droplet

So whenever I answer 'yes' in the interactive prompt then the droplet/server gets added to the list to the `known_hosts`.

Below list of servers in `known_hosts`:

```
bTHm0sge41JacBWrhyqI1LEM7Zq4m86WwqJcCeN1YeaQ10gb0C9wwImtY0S4fhngsf1ZZ2a38hJ0uxeKZHCp5a0vfls+RwTWrAyEs1wWCoDPGHRptm6w2g0
n5n71WhJj9s4EFVe9s7rXDcN183epLdmrD115db67H20s6AskHq+AGgaa/E8G1H6I5scGQp89GAv6DDnRT4Rrrzu6vPYJiNCPiUIVg1V0QPUBxTmcjGHnzK
gQziUx1Cm4t/LQ6hDoek1jo6uaxKQrFHBX50Te1Q17X2rQx79YNHq102mHaMhsBGeZbs28YJox5jG3af5jhST93hAig1V3P2hzdE1N6wP1Cv8BRQNMbWn
WslK3Yubs21HInbVGBMhJyWxNFaBOSY+Pj52p8W72Sx3hbp+RAICFZ/YVB7T5p2AbzqYQ1bEs9d9deh6061Z1Qs=
164.92.225.230 ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldHAyNTYAAAAIbmlldHAyNTYAAABBBF7fTdfj2qjoBVp02JezD/1Bp07jWb5
SWskpbwzarL3v3J7SjTjj6HMDhhViveCPT44a0cEU5FLyUg1cNjssE=
157.230.29.95 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPEjMf+ckvYMALbeqcdiNzDq/czS8k3MHdGUK42EicrI
157.230.29.95 ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCWUk9bKvUOZ7L9+nTR1vFaIuMCHgALoV0/CGN5H0G13s03rx2QKIXgz7dSUvexGX+PN
x853uIriwXt5wy27HuUq22eBZ5zA0InArjF7P+ApvRMI4tc0U1AF51BLu2D1/vN4vPYm546kRcKaoazcDVBe0KCLuycnjCHRJVA4Vdc44YLSFYiQijyQPt
3y+Eg4CCLQisD7HYQ7Dh8RPPK59GON+EuBBZFmPaeNpeMvXSECV30TgW7Z8CprKC3ZHH8vS9moSA4tHX6zL0CzUg7d0RnMwCbjliFjKkIVoSd3XxLqNOy0
Wc98dzW1la81M7TodjSkmtbdkpk7VXDfj8X7iQikYzSL9hFRuNPKhpYfRzAUZdIxxmNJK7qS4vmoCYW+Y4rq500bVPr+J55xd05/2TySwCc/dG16Mif9
cQJaedaSSiSsSCeir0odizYinWFL69oiNvYExsMwrvMqf+q2sU/ap7vs9VZaPwTu/gxWBGnz/5z1z0Hp/s1+5oM4M=
157.230.29.95 ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldHAyNTYAAAAIbmlldHAyNTYAAABBBG/8f8iCpaRft1Tw8fSiEsEP1MFZxvdgir
p9JL7C3XebQ7Q0eM3UDc0EYd7x7NE7k1ZBFJi3kT+6VsNW1gUgXFfa=
157.230.29.113 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII33+bTQcdU9uc2M7050gfHbtkA4+sq0n9oPSNnldwpu
157.230.29.113 ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGC6qjFBSQ/1hbZeTkH+2f9y/yq0CqYsIDtQFIVPb+MwVQiRG2thMD3f1AswvpGQmpu
/c08D4EIw16fvgkxscrIKz9ahcNnZAhIyeZ/n8Ngr3SNH7JV1r4BB690tUytMJ3y5pa+DaG9LuZRgz6gju4fJCRx1EP+b+neXv4XbyJVT2W5qoYQ6syhv
XUeIEUFEmZ1GH18Xp0Wag918vPbBpg0/904rpngZkpPU6cGPAHEgfME6bnxxd6VHaBj55V02YvzExoi5h8LPU32yQciBFUgQLoAGiWwYRByAn7Y0S6X2W6
aFnr1QCZ9y17k/s8k40grDjyMjIc6sWgQpJqz1yJ3WLE/O6FP4PXZG3qDv5Q+3Y3x0tc7rRukmSZQ7IymXamonLiKIVSEqaSjvxyOWP0x7DJBQmvc+By7g
pgU4GTZsUXfEM0WBCeA6wziH50wvE6hCvMZsQt30RjbrPY/kp7kYJWwvF0pxBgaSM8pT5RRdvV8jnInjbCewsLs=
157.230.29.113 ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldHAyNTYAAAAIbmlldHAyNTYAAABBBBC5Z9L4Kyf0HS/lccUaTaowR7HW42WIj
g3691Zaz1dZt08L84Tt4RMKaNSGxalwn68UVHQY9LBkzKtEMB+LCXw=
ec2-18-193-88-102.eu-central-1.compute.amazonaws.com ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGW78fy7UDnG+hmv05yrNsJA87BYQ
nDQEAaYntp5Rz1
ec2-18-193-88-102.eu-central-1.compute.amazonaws.com ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldHAyNTYAAAAIbmlldHAyNT
YAAABBBPnYfabyhWnonphylurZzqM1mvsYyGi5EYtC76Z07up1KnH37MOVUMI4kAjBxLXnc3xGzd9yMrD47GLH54=
ec2-18-192-209-223.eu-central-1.compute.amazonaws.com ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMnzwPBWRT/IdviNnesU7Y2dDVHPh
rsz5GfQ5jUqofAR
ec2-18-192-209-223.eu-central-1.compute.amazonaws.com ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlldHAyNTYAAAAIbmlldHAyN
TYAAABBD3o69Qs3Z/XFas6YwudfV2p18mIfFDg0Kue/rRF+j7A03MH4FnFEQyUDCUJni8ufcPpu1uCs2pVRXaez4244=
nana@macbook /Users/nana
```

So let's make our machine to add the server to the known_hosts list (I did this in the previous chapters to fix my issue as I didn't get the prompt interactive question:

```
-> ssh-keyscan -H 46.101.80.55 >> ~/.ssh/known_hosts
```

-H -> hostname/IP is stored in or hashed

In the previous chapters I used

```
-> ssh-keyscan 46.101.80.55 >> ~/.ssh/known_hosts
```

The difference is whether the hostname/IP is stored in plaintext or hashed in known_hosts — both fetch and trust the same key, just formatted differently.

Without -H (what I ran in previous chapters)

```
46.101.80.55 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA...
```

Plain, human-readable. You can grep 46.101.80.55 ~/.ssh/known_hosts and see it immediately.

With -H

```
[1]X8kJ9f2s...base64salt...[Yk3mP...base64hash... ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAA...
```

The hostname is replaced with an HMAC-SHA1 hash (salted). SSH can still verify a connection against it — it hashes whatever host you're connecting to and compares — but a human reading the file can't tell which entry belongs to which server.

Why it matters

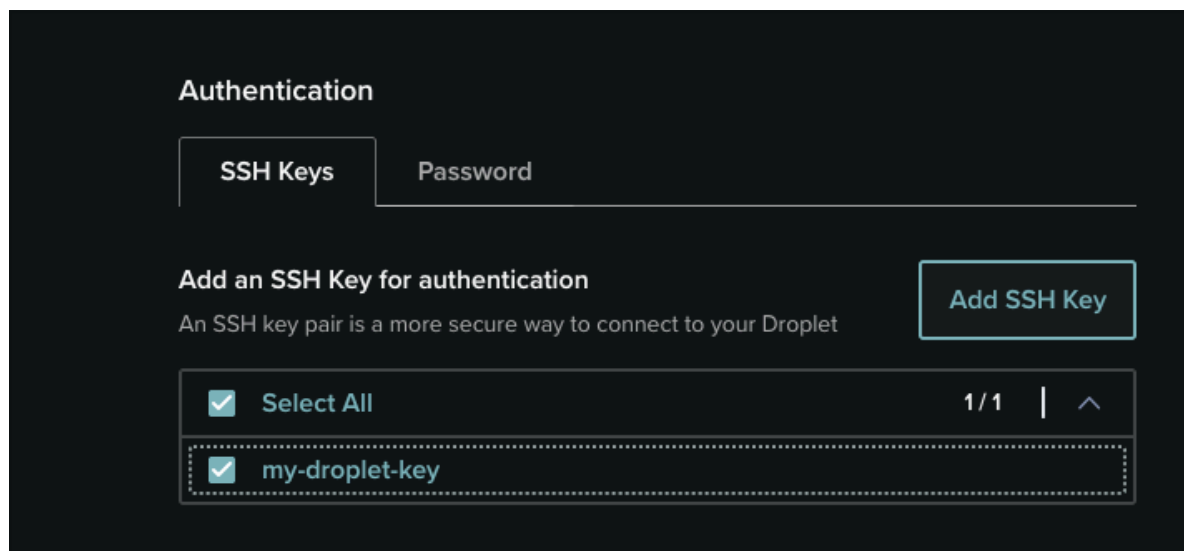
-H is a privacy/security measure: if your known_hosts file ever leaks (backup,

stolen laptop, shared dotfiles repo, etc.), an attacker learns the key fingerprints but not which hosts you connect to — useful if the list of your infrastructure's IPs/hostnames is itself sensitive. Without -H, the file is effectively a directory of every server you've ever trusted.

Trade-off

Hashed entries are harder to manage by hand — you can't grep for an IP to eyeball whether it's there. But this rarely matters day-to-day because the standard tools (ssh-keygen -F <host> to check, ssh-keygen -R <host> to remove) work transparently with hashed entries — they hash your query and compare, so you don't need to read the hash yourself.

B> for the droplet to recognise my control machine the droplet must have the public key stored, which we did at the moment of creating the droplet by selecting the key pair:



We can check this by sshing into the droplet and looking at the ~/.ssh/authorized_keys file:

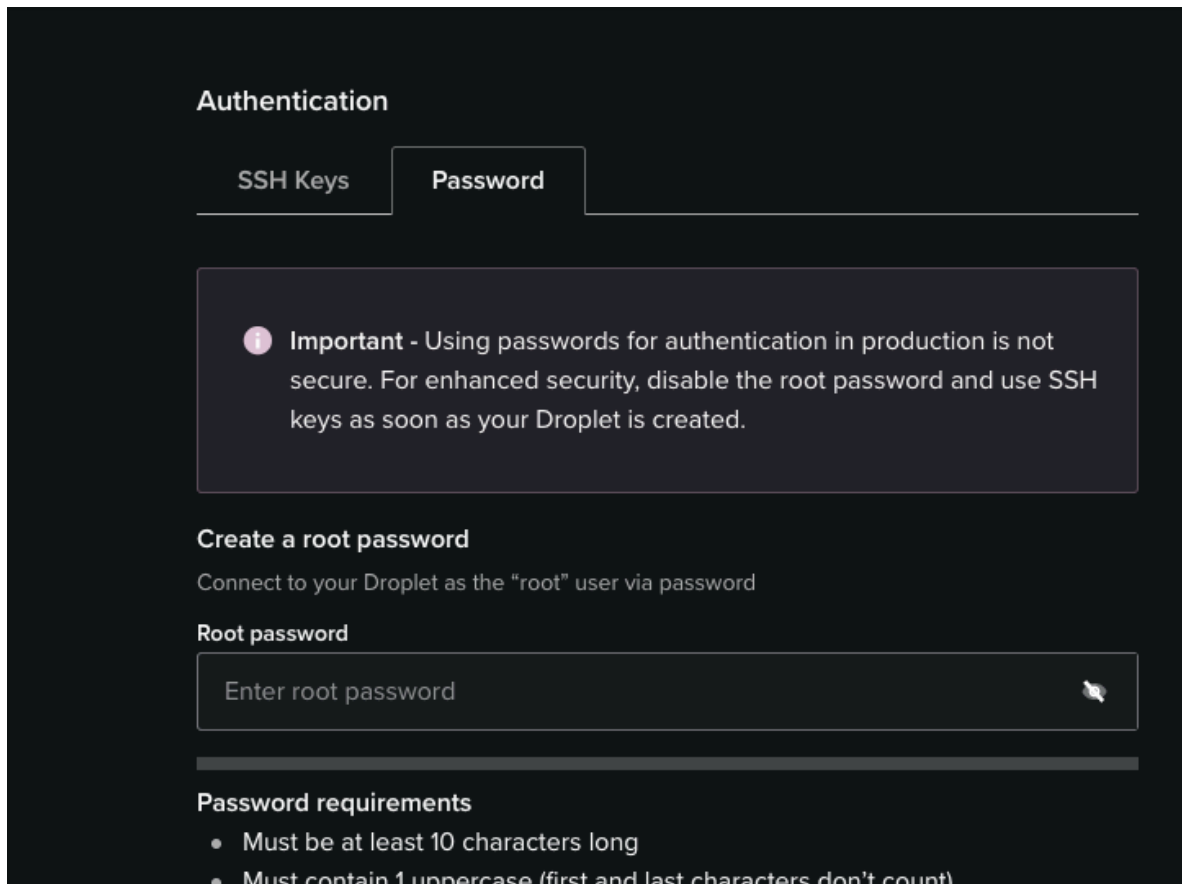
```
Last login: Tue Jul 14 12:09:41 2026 from 140.228.47.252
root@ubuntu-s-1vcpu-2gb-lon1-02:~# cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIx1WUFWNLgXDq/tq7dfqssZLJN0AYqC
root@ubuntu-s-1vcpu-2gb-lon1-02:~# █

X ~ (-zsh)
Last login: Tue Jul 14 11:59:20 on ttys002
> cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIx1WUFWNLgXDq/tq7dfqssZLJN0AYqC
~ █
```

The key from our droplet -> cat ~/.ssh/authorized_keys is the same as the

public key from our machine -> `cat ~/.ssh/id_ed25519.pub`

If I have a droplet without a key pair assigned, for example Nan created a new droplet and instead of an ssh key pair it selected password



The screenshot shows the 'Authentication' section of a DigitalOcean droplet configuration page. It has two tabs: 'SSH Keys' and 'Password', with 'Password' being the active tab. A dark grey informational box contains a warning icon and text: 'Important - Using passwords for authentication in production is not secure. For enhanced security, disable the root password and use SSH keys as soon as your Droplet is created.' Below this, the section 'Create a root password' is shown, with the instruction 'Connect to your Droplet as the "root" user via password'. Under 'Root password', there is a text input field with the placeholder 'Enter root password' and a small eye icon to toggle visibility. At the bottom, 'Password requirements' are listed: 'Must be at least 10 characters long' and 'Must contain 1 uppercase (first and last characters don't count)'.

Authentication

SSH Keys

Password

Important - Using passwords for authentication in production is not secure. For enhanced security, disable the root password and use SSH keys as soon as your Droplet is created.

Create a root password

Connect to your Droplet as the "root" user via password

Root password

Password requirements

- Must be at least 10 characters long
- Must contain 1 uppercase (first and last characters don't count)

Then when we ssh into the droplet we have to provide the password and when we check the `~/.ssh/authorized_keys` file it is empty:


```
% ssh root@178.128.241.118
root@178.128.241.118's password:
"System is booting up. Unprivileged users are not permitted to log in yet. Please, see pam_nologin(8)."
```

"System is booting up. Unprivileged users are not permitted to log in yet. Please, see pam_nologin(8)."

Welcome to Ubuntu 23.10 (GNU/Linux 6.5.0-9-generic x86_64)

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

System information disabled due to load higher than 1.0

0 updates can be applied immediately.

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

```
root@ubuntu-s-1vcpu-2gb-ams3-01:~# cat .ssh/authorized_keys
root@ubuntu-s-1vcpu-2gb-ams3-01:~#
```

Now how can we add a key pair to an existing droplet without it?

-> `ssh-copy-id root@178.128.241.118`

`ssh-copy-id` -> will take the public id from the default location which is `~/.ssh/id_rsa.pub`

```
nana@macbook /Users/nana
% cat ~/.ssh/id_rsa.pub
```

`oot@178.128.241.118` -> And it will copy the public key into the host
`oot@178.128.241.118` in the root user's home directory in the default location
`~/.ssh/authorized_keys`

So we execute:

```
nana@macbook /Users/nana
% ssh-copy-id root@178.128.241.118
```

```
nana@macbook /Users/nana
% ssh-copy-id root@178.128.241.118
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/Users/nana/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
root@178.128.241.118's password:
Number of key(s) added: 1
Now try logging into the machine, with: "ssh 'root@178.128.241.118'"
and check to make sure that only the key(s) you wanted were added.
```

And we can see that 1 key was added! 🙌

Then if we ssh again

-> ssh root@178.128.241.118

No password authentication is needed this time

```
nana@macbook /Users/nana
% ssh root@178.128.241.118
Welcome to Ubuntu 23.10 (GNU/Linux 6.5.0-9-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Mon Dec 18 18:02:25 UTC 2023

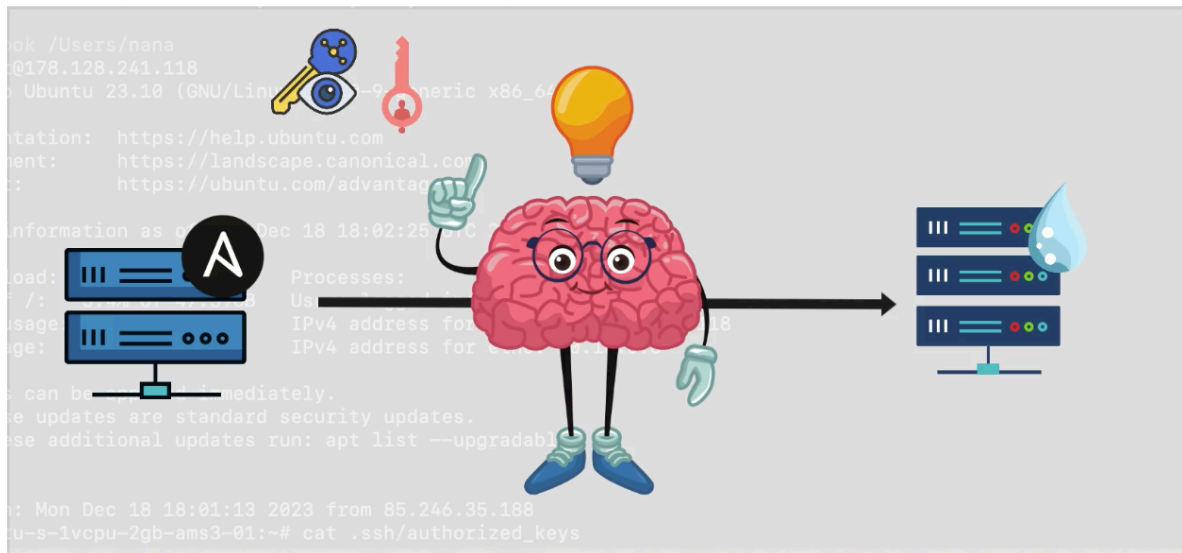
System load:  0.52               Processes:           97
Usage of /:   3.4% of 47.39GB    Users logged in:    0
Memory usage: 11%               IPv4 address for eth0: 178.128.241.118
Swap usage:   0%                IPv4 address for eth0: 10.18.0.6

63 updates can be applied immediately.
30 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Last login: Mon Dec 18 18:01:13 2023 from 85.246.35.188
root@ubuntu-s-1vcpu-2gb-ams3-01:~# ca
```

because the key pair is being used from the ~/.ssh/authorized_keys

```
root@ubuntu-s-1vcpu-2gb-ams3-01:~# cat .ssh/authorized_keys
root@ubuntu-s-1vcpu-2gb-ams3-01:~#
```



So the above was how we can get the long-live servers to identify with our control machine and vice versa.

Now let's disable the Host Key checking

Disable Host Key Checking

Disabling the Host key check is not ideal due to security but it makes sense when working with ephemeral infrastructure, for example running test and then destroying the server once tests are done or when scaling up and down servers due to traffic for an application

Ephemeral Infrastructure

- Servers are dynamically created and destroyed



For ephemeral servers it is inconvenient to do the Host key check.

To disable the host key check we do it in ansible's configuration file:

In older ansible versions ansible created a default file in etc:

-> /etc/ansible

```
nana@macbook /Users/nana
% ls /etc/ansible
ls: /etc/ansible: No such file or directory
nana@macbook /Users/nana
%
```

But in the recent versions that directory is not getting created and we don't have the configuration file, so we will create one ourselves 💡

And we will create the file in one of the default locations where ansible will pick it up:

Config File Default Locations

- ▶ */etc/ansible/ansible.cfg*
- ▶ *~/.ansible.cfg*

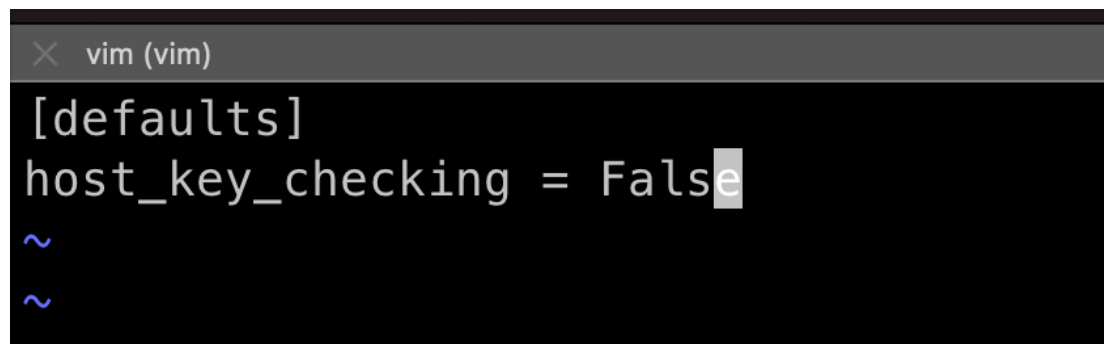
So let's create it in the user's home directory (we can see in the ansible docs all the configuration we can work with)

-> vim ~/.ansible.cfg

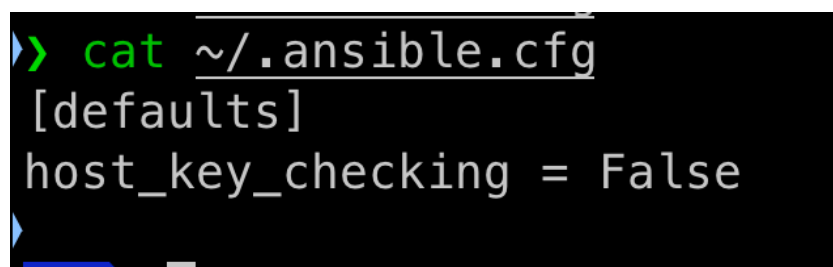
And we add this configuration:

[defaults]

host_key_checking = False



```
vim (vim)
[defaults]
host_key_checking = False
~
~
```



```
> cat ~/.ansible.cfg
[defaults]
host_key_checking = False
```

For testing I will add a new droplet to DigitalOcean and add it to the 'hosts' file

Search by resource name or public IP (Ctrl+F)

Create

My team

Backups

☐ **Enable automated backups**
Restore your Droplet quickly if it fails. Configure backup frequency and retention. With usage-based pricing, pay only for the backup's restorable file size.

Authentication

SSH Keys

Password

Add an SSH Key for authentication

An SSH key pair is a more secure way to connect to your Droplet

Add SSH Key

Select All

1/1

my-droplet-key

Networking

Public IP addresses are optional. A private IPv4 address is always assigned for secure internal traffic within the same VPC.

☒ **Public IPv4 address**
Allows your Droplet to send and receive traffic from the public internet. Disable to restrict this Droplet to your VPC.

☐ **Public IPv6 address**
Built-in authentication and privacy support, and makes it difficult for hackers to launch scanning attacks.

Summary

Compute

Plan Type: Basic
CPU Option: Regular
vCPU: 1
RAM: 2 GB
Disk: 50 GB
Bandwidth: 2 TB
Slug: s-1vcpu-2gb

\$12.00/mo

Total cost

\$12.00/month

\$0.018/hour

Create Droplet

Want to maximize efficiency and cost?

Programmatically manage Droplets in a repeatable and re-usable way with our API. Create via API

Name	IP Address	Created ▲	Tags
ubuntu-s-1vcpu-2gb-lon1 2 GB / 50 GB / LON1 - Ubuntu 24.04 (LTS) x64	159.65.25.34	28 seconds ago	—
ubuntu-s-1vcpu-2gb-lon1-02 2 GB / 50 GB / LON1 - Ubuntu 24.04 (LTS) x64	46.101.88.242	5 hours ago	—
ubuntu-s-1vcpu-2gb-lon1 2 GB / 50 GB / LON1 - Ubuntu 24.04 (LTS) x64	46.101.80.55	5 hours ago	—
jenkins-server 4 GB / 80 GB / LON1 - Ubuntu 24.04 (LTS) x64	165.22.116.21	7 months ago	—

```

vim (vim)
[droplet]
46.101.88.242
46.101.80.55
159.65.25.34

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root
~
~

```

```

> vim hosts
> cat hosts
[droplet]
46.101.88.242
46.101.80.55
159.65.25.34

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root

```

Now let's test it

-> ansible droplet -i hosts -m ping

```

> ansible droplet -i hosts -m ping
[WARNING]: Host '46.101.80.55' is using the discovered Python interpreter at '/usr/bin/python3.12',
other Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible_appendices/interpreter_discovery.html for more information.
46.101.80.55 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '46.101.88.242' is using the discovered Python interpreter at '/usr/bin/python3.12',
other Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible_appendices/interpreter_discovery.html for more information.
46.101.88.242 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '159.65.25.34' is using the discovered Python interpreter at '/usr/bin/python3.12',
other Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible_appendices/interpreter_discovery.html for more information.
159.65.25.34 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}

```

And it works, the host key check is disabled!

NO SSH key check was made.

With `host_key_checking = False`, Ansible passes `StrictHostKeyChecking=no` to SSH under the hood. What that actually does:

- The check is skipped — SSH will connect to any host regardless of whether its key matches (or exists) in `known_hosts`. No prompt, no failure.
- `known_hosts` still gets written to — SSH still silently appends the host's key to

~/.ssh/known_hosts on first connect, it just doesn't ask you to confirm it first, and doesn't verify it against anything already there.

So the file isn't skipped, just the enforcement is. And that's actually the more important nuance: it's not just "no check the first time" — it means no check ever, on every single connection, forever, for every host matched by that config. If a host's key ever legitimately changes (server rebuilt) or illegitimately changes (someone intercepting the connection), Ansible will silently accept the new key and carry on either way, with no warning.

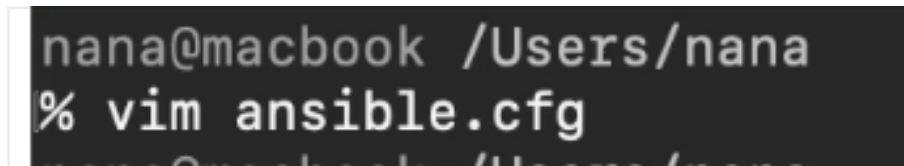
That's the real trade-off versus the ssh-keyscan approach from earlier: ssh-keyscan trusts a host once, after which normal verification resumes and would catch a future key change. `host_key_checking = False` disables verification permanently, for every host, including ones whose keys change later for suspicious reasons.

-> `.ansible.cfg` will apply for all the ansible projects, it is linked to my home directory. So I can execute ansible commands from any directory and it will execute `.ansible.cfg`

If I want the configuration to only apply to one project, let's say the project lives in `/User/astrid`

Then I create the config file without being hidden

-> `ansible.cfg`

A terminal window with a dark background. The prompt is `nana@macbook /Users/nana`. The user has entered `% vim ansible.cfg`. The next line shows the prompt `nana@macbook /Users/nana` again.

```
nana@macbook /Users/nana
% vim ansible.cfg
nana@macbook /Users/nana
```

- `~/.ansible.cfg` (home directory) acts as a fallback used from any directory that doesn't have its own closer config.
- A local `ansible.cfg` (non-hidden) in a specific directory takes priority over the home one.

The configuration file

Changes can be made and used in a configuration file which will be searched for in the following order:

- ```
thon": •
```
- `ANSIBLE_CONFIG` (environment variable if set)
  - `ansible.cfg` (in the current directory)
  - `~/.ansible.cfg` (in the home directory)
  - `/etc/ansible/ansible.cfg`

```
thon
```

Ansible will process the above list and use the first file found, all others are ignored.

So the configuration will apply to only to ansible commands that I execute from the Users/astrid

it's "first found wins, entirely." Ansible doesn't combine settings from both files. If it finds a local `ansible.cfg` in your current directory, it uses only that file — every setting in it, and nothing from `~/.ansible.cfg`, even settings the local file doesn't mention. Those just fall back to Ansible's built-in defaults, not to your home config.

This matters concretely for you: you just set `host_key_checking = False` in `~/.ansible.cfg`. If you now create a project-level `ansible.cfg` and don't repeat that line, host key checking will be back on whenever you run from that project directory — not inherited from home.

2. It's tied to your exact current working directory, not "the project." Ansible checks only the directory you're actually running the command from — it doesn't search upward through parent folders the way git finds `.git`. So if your project lives at `/Users/astrid` and you `cd` into a subfolder (e.g. `/Users/astrid/playbooks`) and run ansible from there, it will not find `/Users/astrid/ansible.cfg` — you'd need a copy in that subfolder too, or run commands from the project root.

So if I have both `.ansible.cfg` and `ansible.cfg` in the same dir like my home dir then When I run Ansible from that directory, the non-hidden one wins outright — my `~/.ansible.cfg` becomes completely inactive whenever I am in that directory.

And if I create a new directory and run ansible there and I don't create a `ansible.cfg` it will read the `.ansible.cfg` from the home dir.

Ansible's lookup order is: `ANSIBLE_CONFIG` env var → `ansible.cfg` in your current directory → `~/.ansible.cfg` → `/etc/ansible/ansible.cfg` (system default) — checked in that order, first one found wins.

So a brand-new directory with no local `ansible.cfg` skips straight past the second step and falls through to `~/.ansible.cfg` — same behavior as running

from anywhere else that doesn't have its own config. Your `host_key_checking = False` setting would apply there.